Architecture and Operations

Обновлено 29 мая 2026, 00:59 Андрей Коновалов

Backend Modules

Module	Responsibility
AuthModule	registration, login, refresh, logout, JWT strategies
UsersModule	user profile and live tournament recovery endpoint
TournamentsModule	tournaments, rounds, submissions, votes, realtime events
RedisModule	ioredis wrapper
FingerprintModule	device fingerprint middleware/decorator

Tournaments Services

Service	Responsibility
TournamentService	create, join, leave, cancel, start
GetDraftTournamentQueryService	list/details draft tournaments
GetFullTournamentQueryService	participant-only full snapshot
RoundPromptService	random bilingual prompt selection without repeats
RoundSubmissionPhaseService	submission deadline, progress, phase completion
RoundVotingFlowService	sequential voting, vote finalization, timeout votes
RoundCompletionService	round results, leaderboard, next round, tournament finish

RoundResultCalculatorService	round result calculation
TournamentRoundSubmissionService	submission validation and upsert
TournamentRoundVoteService	vote validation and upsert
TournamentEventsService	typed event facade, enqueue realtime jobs
TournamentSocketEmitterService	low-level Socket.IO room emit/eject
TournamentPresenceService	Redis-backed presence
TournamentRoomAccessService	room access validation
WsJwtAuthService	Socket.IO JWT handshake auth

Realtime Event Delivery

Current implementation:

1. A domain service calls `TournamentEventsService`.
2. `TournamentEventsService` enqueues a BullMQ job in the `tournaments` queue.
3. `TournamentEventsProcessor` consumes the job.
4. The processor calls `TournamentSocketEmitterService`.
5. The socket emitter broadcasts to room `tournament:{tournamentId}` or ejects user sockets.

Queue job names:

Job	Purpose
broadcast-event	broadcast server-to-client event
eject-from-room	emove user sockets from tournament room

Default queue options:

- attempts: 3;
- exponential backoff starting at 500 ms;
- keep up to 1000 completed jobs;
- keep up to 1000 failed jobs.

Phase Deadlines

Submission and voting deadlines are scheduled in memory:

- `SubmissionPhaseDeadlineRegistryService`;
- `VotingDeadlineRegistryService`.

On application bootstrap:

- submission rounds are loaded and scheduled again;
- open voting rounds are loaded and scheduled again.

This is acceptable for an MVP with one backend instance, but it is not a full distributed scheduler.

PostgreSQL Consistency

The code uses database transactions and pessimistic locks for critical state transitions.

Examples:

- tournament creation locks owner participation via `pg_advisory_xact_lock(hashtext(userId))`;
- tournament start locks the tournament row and checks participants;
- submission upsert locks the round row;
- vote upsert locks the round row;
- voting finalization locks the round row;
- round completion locks round and tournament rows.

Active participation conflicts are guarded by advisory locks and policy checks.

Environment Variables

Variable	Required	Default	Description
APP_NODE_ENV	yes	-	app environment
APP_PORT	no	3001	HTTP port
APP_SWAGGER_ENABLED	yes	-	enable Swagger/AsyncAPI helpers
APP_CORS_ORIGINS	yes	-	allowed origins list
JWT_SECRET	yes	-	JWT signing secret
JWT_EXPIRES	yes	-	access token TTL seconds
JWT_REFRESH_EXPIRES	yes	-	refresh token TTL seconds
REDIS_URL	no	-	Redis connection URL
REDIS_HOST	no	-	Redis host
REDIS_PORT	no	6379	Redis port
REDIS_PASSWORD	no	-	Redis password
DB_URL	no	-	PostgreSQL connection URL
DB_HOST	no	-	PostgreSQL host
DB_PORT	no	5432	PostgreSQL port
DB_DATABASE	no	-	database name
DB_USERNAME	no	-	database user

DB_PASSWORD	no	-	database password

Local Scripts

Command	Purpose
npm run start:dev	start NestJS in watch mode
npm run start:debug	start with debug and watch mode
npm run build	compile TypeScript
npm run start:prod	run compiled app
npm run docker:up	start PostgreSQL and Redis via Docker Compose
npm run lint	ESLint with auto-fix
npm run format	Prettier formatting
npm run mig:create-win --name=nam	create empty migration on Windows
npm run mig:generate-win --name=name	generate migration on Windows
npm run mig:run	run migrations
npm run mig:revert	revert last migration
npm run mig:show	show migration status

API Documentation Endpoints

If enabled by configuration:

- Swagger UI: `/api`;
- AsyncAPI UI/spec: `/api-ws`.

Deployment Notes

MVP assumptions:

- one backend instance;
- Redis is available for refresh tokens, presence, and BullMQ;
- PostgreSQL is available for persistent domain state;
- no distributed Socket.IO adapter;
- no durable phase scheduler beyond bootstrap recovery.